# CargoVibe

Architecture & Technical Decisions

---

## What Was Built

A fullstack monorepo for managing truck parking requests, consisting of three layers:

- **Backend** — Azure Functions v4 (TypeScript) running locally via an Express adapter
- **Mobile** — Expo React Native app with two screens (list + detail), supporting both web and native
- **AI integration** — Natural language assistant via the Claude API, added as an optional enhancement after all core requirements were completed

*The AI assistant was not required by the brief. Core requirements were completed first; the AI feature was added afterwards to explore how operational parking data could be queried using natural language.*

---

## Backend Decisions

### Azure Functions v4 with Express Adapter for Local Development

To simplify onboarding and ensure the project can be run immediately without additional tooling, a lightweight Express adapter was introduced for local development. The handlers themselves use the real AF v4 signature (`HttpRequest`, `InvocationContext`, `Promise<HttpResponseInit>`) unchanged. In a real Azure deployment the adapter is removed and the cloud runtime takes over — nothing else changes.

```
Local:   Browser → Express adapter → AF handler → Repository
Azure:   Browser → Azure runtime   → AF handler → Repository
```

### Functional Style Over Classes

The repository layer uses plain exported functions (`findAll`, `findById`, `create`, `updateStatus`, `deleteById`) with module-scoped state. The data layer has no behaviour that benefits from class encapsulation. A `clearStore()` function provides test isolation without needing to instantiate a fresh object per test.

Error classes (`NotFoundError`, `InvalidTransitionError`, etc.) remain as classes because `instanceof` checks in the error handler require them — that is the one place where class inheritance earns its keep.

### Status Machine as a Data Table

The entire lifecycle is encoded as a lookup table rather than a chain of conditionals. This makes the rules immediately visible, trivially testable, and easy to extend.

```
const STATUS_TRANSITIONS = {
  pending:     ['approved', 'rejected'],
  approved:    ['checked_in'],
  checked_in:  ['checked_out'],
  rejected:    [],
  checked_out: [],
}
```

The same transition model is used in both the backend and mobile layers so the UI only presents valid actions, while the backend remains the ultimate source of truth and performs all validation. Changing a transition is a one-line edit with no risk of breaking other paths.

### In-Memory Store

An in-memory `Map` was chosen because persistence was not required by the brief.

| Trade-offs | |
|---|---|
| + Faster implementation | – Data resets on application restart |
| + Simpler testing (no DB setup) | – No user roles or permissions |

*For production this would be replaced with a SQLite or PostgreSQL implementation behind the same repository interface — no routes, tests, or handlers would change.*

### HTTP Status Codes

Status codes were chosen deliberately to communicate the precise failure mode:

| Situation | Code | Rationale |
|---|---|---|
| Missing or invalid fields | 400 | Request is malformed |
| ID not found | 404 | Resource does not exist |
| Invalid status transition | 422 | Request is valid JSON but violates business rules |
| Modifying a terminal state | 409 | Conflict with current resource state |

*422 is deliberate: the request body is well-formed JSON and the status value is valid, but the business rule blocks it. That is exactly what 422 Unprocessable Entity is for. Using 400 here would be incorrect.*

---

## Mobile Decisions

### Expo over Bare React Native

The brief requires web and native support. Expo's managed workflow handles both with `npm run web` and `npm start`. No Xcode or Android Studio is needed to run or demo the app.

### Custom Hooks for State Management

`useParkingRequests` (list) and `useParkingRequest` (detail) handle all API state with plain `useState` and `useCallback`. Two screens with independent data needs do not require a global state manager — introducing Redux or Zustand here would be over-engineering.

### Note Field Excluded from Detail Screen

The brief explicitly states Screen 2 shows all fields except note. This is implemented exactly as specified.

---

## Testing

| Layer | Tests | Approach |
|---|---|---|
| Repository unit | 22 | Pure function calls, no HTTP |
| AF handler unit | 14 | Direct function calls with mock request objects |

Testing was focused on business logic and API behaviour because those areas carry the highest risk of defects. Azure Function handlers are pure async functions — they accept a request object and return a response object — which means they can be tested by calling them directly with no HTTP server, no supertest, and no port binding.

*UI testing was intentionally omitted given the time budget and the evaluation criteria prioritising architecture and functionality over end-to-end coverage.*

---

## AI Integration

`POST /api/ai/chat` proxies the Claude API server-side. Two reasons: the API key never reaches the mobile bundle, and the server injects a live snapshot of all parking requests as context so the AI can answer questions about real data.

Responses include a `suggestedAction` field the mobile app can use to trigger status updates — but every suggested action still goes through the normal endpoint and the same validation rules. The AI cannot bypass business logic.

---

## Deliberately Out of Scope

To stay within the time budget and focus on the evaluation criteria, the following were intentionally not implemented:

- Authentication and authorisation
- Role-based permissions
- Persistent database storage
- Push notifications
- Offline support
- End-to-end testing

The focus was on delivering the required functionality with clean architecture, enforced business rules, and testable code.

---

## Trade-offs & Limitations

Known limitations of the current implementation:

- Data is stored in-memory and resets when the application restarts
- No user roles or permissions — any request can approve or reject a booking
- The mobile app relies entirely on network connectivity

---

## What I Would Add Next

If given two additional days, the priority order would be:

- **SQLite or PostgreSQL persistence** — `better-sqlite3`; the repository layer is already designed for this swap
- **Schema-based validation with Zod** — stricter input contracts across all endpoints
- **Integration tests** covering the complete request lifecycle end-to-end
- **JWT authentication and role-based permissions** — status changes should not be available to every user
- **Optimistic UI updates** — update the mobile UI immediately and roll back on error
- **Push notifications** — alert drivers when their request status changes